

Guía docente

Colecciones de datos I: Tuplas, Conjuntos y Diccionarios

Curso: Herramientas Computacionales

1. Información general

Tema	Colecciones de datos I: tuplas, conjuntos, diccionarios, aplicaciones y diferencias entre estructuras.
Duración	2 horas (120 minutos).
Prerrequisitos	Variables, tipos de datos básicos, entrada/salida, condicionales y ciclos <code>while/for</code> .
Material principal	Notebook <code>Colecciones_Datos_I_Tuplas_Conjuntos_Diccionarios.ipynb</code> .
Enfoque	Concepto → ejemplo → ejecución → pregunta → aplicación.

2. Propósito pedagógico

La finalidad de esta clase no es que los estudiantes memoricen tres nuevas sintaxis. El objetivo principal es que empiecen a desarrollar el criterio de **seleccionar una estructura de datos de acuerdo con el problema**.

Los estudiantes ya conocen variables, condicionales y ciclos. Por ello, la clase debe aprovechar esos conocimientos para mostrar que una colección permite procesar muchos datos de manera sistemática.

La pregunta transversal de la clase debe ser:

¿Qué estructura representa mejor la información que tengo y las operaciones que necesito realizar?

3. Objetivos de aprendizaje

Al finalizar la sesión, el estudiante debería poder:

1. Crear y consultar tuplas.
2. Explicar qué significa que una tupla sea ordenada e inmutable.
3. Recorrer una tupla mediante un ciclo `for`.
4. Crear conjuntos y comprender que sus elementos son únicos.
5. Realizar unión, intersección y diferencia entre conjuntos.
6. Crear y modificar diccionarios mediante pares clave-valor.
7. Recorrer diccionarios con `keys()`, `values()` e `items()`.
8. Diferenciar tuplas, conjuntos y diccionarios.
9. Elegir una estructura apropiada para un problema sencillo.

4. Distribución de los 120 minutos

Momento	Tiempo	Propósito
Inicio	10 min	Conectar colecciones con variables y ciclos.
Tuplas	25 min	Concepto, índices, inmutabilidad, recorrido y desempaqueado.
Conjuntos	25 min	Unicidad, pertenencia y operaciones entre conjuntos.
Diccionarios	30 min	Clave-valor, consulta, modificación y recorrido.
Comparación e integración	20 min	Elegir estructuras y resolver un caso.
Cierre	10 min	Mini-reto y comprobación conceptual.

5. Estrategia general de la clase

Para cada estructura se recomienda seguir la misma secuencia:

1. Presentar una necesidad concreta.
2. Mostrar qué problema aparece si usamos variables independientes.
3. Introducir la estructura.
4. Ejecutar un ejemplo pequeño.
5. Preguntar a los estudiantes qué esperan que ocurra antes de ejecutar.
6. Modificar el código en vivo.
7. Conectar la estructura con un ciclo `for`.
8. Compararla con las otras estructuras.

Este patrón evita que la clase se convierta en una lista de métodos que los estudiantes deben memorizar.

6. Guion docente detallado

6.1 Inicio: ¿por qué necesitamos colecciones? — 10 minutos

Idea que debe quedar clara: una colección permite agrupar varios datos bajo una misma variable.

Comenzar con:

```
nota1 = 4.0
nota2 = 3.5
nota3 = 4.8
nota4 = 2.9
```

Preguntar:

Si tengo 1000 estudiantes, ¿quiero crear 1000 variables?

La respuesta esperada es no.

Mostrar la idea:

```
notas = [4.0, 3.5, 4.8, 2.9]
```

Aclarar que Python ofrece diferentes colecciones porque no todos los problemas tienen las mismas características.

Pregunta para los estudiantes:

¿Qué diferencia habría entre guardar una lista de nombres, un conjunto de nombres sin repetir y la información completa de un estudiante?

No es necesario resolver inmediatamente la pregunta. Debe quedar como hilo conductor.

6.2 Tuplas — 25 minutos

Concepto

Presentar:

```
coordenadas = (10, 20)
```

Explicar palabra por palabra:

- **coordenadas**: nombre de la variable.
- **=**: asignación.
- **(10, 20)**: literal de una tupla.
- Los valores 10 y 20 son elementos.

Pregunta: ¿cuántos elementos tiene la tupla?

Respuesta: dos.

Índices

Mostrar:

```
frutas = ("manzana", "pera", "uva")
print(frutas[0])
```

Dibujar en el tablero:

índice	0	1	2
valor	manzana	pera	uva

Enfatizar que Python comienza a contar índices desde cero.

Pregunta frecuente: “¿Por qué no empieza en 1?”

Respuesta docente: en Python los índices representan desplazamientos desde el primer elemento. El primer elemento tiene desplazamiento cero.

Inmutabilidad

Mostrar:

```
punto = (3, 5)
# punto[0] = 10
```

Explicar que la línea comentada produciría un **TypeError**.

Pregunta frecuente: “Entonces, ¿una tupla nunca puede cambiar?”

Respuesta: no podemos modificar directamente sus elementos. Si necesitamos otra tupla, podemos crear una nueva.

Ejemplo conceptual:

```
punto = (3, 5)
punto = (10, 5)
```

Aquí no modificamos la tupla original: reasignamos la variable a otra tupla.

Recorrido

Conectar explícitamente con la clase anterior:

```
for fruta in frutas:
    print(fruta)
```

Explicar:

1. `for` inicia un ciclo.
2. `fruta` es la variable que recibirá cada elemento.
3. `in frutas` indica que se recorrerá la tupla.
4. El bloque indentado se ejecuta una vez por elemento.

Desempaquetado

Mostrar:

```
punto = (4, 7)
x, y = punto
```

Preguntar:

¿Qué valor tendrá `x`? ¿Y qué valor tendrá `y`?

Respuesta: `x = 4`, `y = 7`.

Explicar que Python asigna cada elemento a una variable siguiendo el orden.

6.3 Conjuntos — 25 minutos

Motivación

Mostrar:

```
usuarios = {"Ana", "Luis", "Ana", "Carlos", "Luis"}
```

Preguntar:

¿Cuántas personas distintas debería representar este conjunto?

Respuesta: tres.

Explicar que un conjunto representa elementos únicos.

Conjunto vacío

Este punto merece énfasis porque es una fuente frecuente de errores:

```
vacio = {}
print(type(vacio))
```

El resultado es `dict`.

Para un conjunto vacío:

```
vacio = set()
```

Pregunta frecuente: “¿Por qué Python usa dos formas distintas?”

Explicar que las llaves se reservaron para representar estructuras de pares clave–valor cuando no hay elementos. Por ello, Python necesita `set()` para indicar explícitamente un conjunto vacío.

Operaciones

Usar:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)
print(A & B)
print(A - B)
print(A ^ B)
```

Explicar cada operador:

- |: unión.
- &: intersección.
- -: diferencia.
- ^: diferencia simétrica.

Una buena estrategia es pedir primero una predicción.

Pregunta:

¿Qué números aparecen en ambos conjuntos?

Respuesta: 3 y 4.

Luego:

Entonces, ¿qué debería producir `A & B`?

Respuesta: {3, 4}.

Conexión aplicada: comparar estudiantes inscritos en dos actividades.

6.4 Diccionarios — 30 minutos

Motivación

Plantear:

Tengo un estudiante llamado Ana, tiene 20 años y pertenece al curso de Python. ¿Cómo puedo guardar toda esa información de manera organizada?

Mostrar:

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "curso": "Python"
}
```

Explicar la estructura:

clave $\longrightarrow$ valor

Por ejemplo:

"nombre" $\longrightarrow$ "Ana"

Acceso

Mostrar:

```
print(estudiante["nombre"])
```

Contrastar con una tupla:

```
punto[0]
```

En una tupla preguntamos “¿qué hay en la posición 0?”.

En un diccionario preguntamos “¿qué valor corresponde a la clave nombre?”.

Esta diferencia conceptual es muy importante.

Modificar y agregar

```
estudiante["edad"] = 21
estudiante["programa"] = "Ingeniera"
```

Explicar que:

- si la clave existe, se actualiza el valor;
- si no existe, se crea una nueva pareja clave-valor.

`get()`

Mostrar:

```
print(estudiante.get("correo"))
print(estudiante.get("correo", "No registrado"))
```

Explicar que:

```
estudiante["correo"]
```

puede producir `KeyError` si la clave no existe, mientras que `get()` permite definir un valor alternativo.

Recorrido con `items()`

Este es uno de los códigos centrales de la clase:

```
for clave, valor in estudiante.items():
    print(clave, "->", valor)
```

Explicar línea por línea:

1. `for`: inicia la repetición.
2. `clave, valor`: dos variables reciben los dos componentes de cada elemento.
3. `in`: indica la colección a recorrer.
4. `estudiante.items()`: produce pares clave-valor.
5. `print`: muestra la información.

Pregunta frecuente: “¿Por qué necesitamos dos variables después de `for`?”

Respuesta: porque cada elemento producido por `items()` contiene dos componentes: una clave y su valor.

6.5 Comparación de estructuras — 20 minutos

Presentar la siguiente tabla y discutirla:

	Tupla	Conjunto	Diccionario
Orden/posición	Sí	No debe asumirse	No se usa índice
Duplicados	Sí	No	Claves no
Modificable	No	Sí	Sí
Acceso típico	Índice	Pertenencia	Clave
Uso principal	Datos fijos	Valores únicos	Clave-valor

Actividad oral rápida:

Preguntar uno por uno:

1. “Quiero representar (x, y) .”
Respuesta: tupla.
2. “Quiero saber qué estudiantes están inscritos en ambas actividades.”
Respuesta: conjunto.
3. “Quiero guardar el precio de cada producto.”
Respuesta: diccionario.
4. “Quiero eliminar duplicados de una colección.”
Respuesta: conjunto.
5. “Quiero guardar nombre, edad y programa de un estudiante.”
Respuesta: diccionario.

6.6 Ejemplo integrador — 10 minutos

Usar:

```
curso_A = {"Ana", "Luis", "Maria", "Carlos"}
curso_B = {"Luis", "Maria", "Sofia", "Juan"}

notas = {
    "Ana": 4.5,
    "Luis": 3.8,
    "Maria": 4.2,
    "Carlos": 2.9,
    "Sofia": 4.8,
    "Juan": 3.4
}
```

Explicar la elección:

- Los cursos son conjuntos porque interesa la pertenencia y no queremos duplicados.
- Las notas son un diccionario porque relacionamos estudiante con nota.

Luego:

```
for estudiante, nota in notas.items():
    if nota >= 3.0:
        estado = "Aprobo"
    else:
        estado = "No aprobo"
```

```
print(f"{estudiante}: {nota} - {estado}")
```

Este ejemplo integra **diccionarios** + **ciclos** + **condicionales**, reforzando conocimientos anteriores.

7. Mini-reto de cierre

Plantear el problema de los productos:

```
precios = {  
    "pan": 2500,  
    "leche": 4200,  
    "huevos": 15000,  
    "arroz": 6000  
}
```

Pedir que los estudiantes completen:

1. recorrido de productos;
2. recorrido de precios;
3. recorrido de pares producto-precio;
4. consulta mediante `input()`;
5. manejo de producto inexistente.

Antes de escribir código, pedir tres decisiones:

1. ¿Cuál es la clave?
2. ¿Cuál es el valor?
3. ¿Cómo verificamos que un producto existe?

La solución del notebook utiliza `get()` y un condicional.

8. Preguntas frecuentes y respuestas docentes

Pregunta 1: “¿Cuál es la diferencia entre una tupla y una lista?”

Respuesta sugerida:

Ambas permiten almacenar una secuencia y utilizar índices, pero una lista es modificable y una tupla no. En esta clase interesa que el estudiante entienda que una tupla es apropiada cuando los datos representan una colección que conceptualmente no debe modificarse.

Pregunta 2: “¿Un conjunto puede tener listas dentro?”

Respuesta sugerida:

No directamente. Los elementos de un conjunto deben ser objetos que puedan utilizarse como elementos del conjunto, por lo que una lista, al ser mutable, no puede ser un elemento de un `set`. Para esta clase no es necesario profundizar en hashabilidad; basta con conocer la regla práctica.

Pregunta 3: “¿Un diccionario puede tener dos veces la misma clave?”

Respuesta:

No como dos entradas independientes. Si una clave aparece nuevamente, su valor se actualiza.

```
datos = {"a": 10, "a": 20}
```

El resultado efectivo es:

```
{"a": 20}
```

Pregunta 4: “¿Puedo acceder a un conjunto con [0]?”

Respuesta:

No. Un conjunto no se utiliza mediante índices. Si la pregunta es si un elemento pertenece al conjunto, usamos `in`.

Pregunta 5: “¿Por qué usar un conjunto si ya tengo una lista?”

Respuesta:

Porque si el problema requiere valores únicos o comparaciones entre grupos, el conjunto representa mejor la información y ofrece operaciones como unión e intersección.

Pregunta 6: “¿Puedo guardar una lista como valor de un diccionario?”

Respuesta:

Sí. Un valor de un diccionario puede ser prácticamente cualquier objeto apropiado para el problema.

Ejemplo:

```
estudiante = {  
    "nombre": "Ana",  
    "materias": ["Python", "Matematicas"]  
}
```

No es necesario profundizar en estructuras anidadas si el tiempo es limitado; puede mencionarse como extensión.

Pregunta 7: “¿Por qué `get()` es mejor que usar corchetes?”

Respuesta:

No siempre es “mejor”. Depende de lo que necesitemos. Los corchetes son apropiados cuando sabemos que la clave existe y queremos detectar un error si no existe. `get()` es útil cuando la ausencia de una clave es un caso esperado.

9. Errores conceptuales que se deben vigilar

1. Confundir `{}` con un conjunto vacío.
2. Creer que un conjunto conserva una posición estable para cada elemento.
3. Intentar modificar una tupla.
4. Usar índices para acceder a un diccionario.
5. Confundir clave con valor.
6. Pensar que `items()` devuelve solo valores.
7. No distinguir entre “elemento duplicado” y “clave duplicada”.
8. Elegir estructuras por sintaxis y no por el problema.

10. Descripción del código principal del notebook

10.1 Tupla

```
frutas = ("manzana", "pera", "uva")

for fruta in frutas:
    print("Fruta:", fruta)
```

Funcionamiento:

1. Se crea la tupla.
2. El ciclo solicita cada elemento.
3. La variable `fruta` toma sucesivamente los valores.
4. `print()` muestra cada elemento.

10.2 Conjunto

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)
print(A & B)
print(A - B)
```

Funcionamiento:

- `A | B`: combina los elementos de ambos conjuntos.
- `A & B`: conserva los elementos comunes.
- `A - B`: conserva los elementos que están en A pero no en B.

10.3 Diccionario

```
estudiante = {
    "nombre": "Ana",
    "edad": 20,
    "curso": "Python"
}

for clave, valor in estudiante.items():
    print(clave, "->", valor)
```

Funcionamiento:

1. Se crea un diccionario.
2. Cada entrada contiene una clave y un valor.
3. `items()` permite recorrer ambas partes.
4. En cada iteración, `clave` recibe la clave y `valor` recibe el valor.

10.4 Ejemplo integrador

```
notas = {
    "Ana": 4.5,
    "Luis": 3.8,
    "Maria": 4.2,
```

```
"Carlos": 2.9
}

for estudiante, nota in notas.items():
    if nota >= 3.0:
        estado = "Aprobo"
    else:
        estado = "No aprobo"

    print(f"{estudiante}: {nota} - {estado}")
```

Este fragmento es pedagógicamente importante porque integra tres conceptos:

1. **Diccionario:** almacena la relación estudiante–nota.
2. **Ciclo:** procesa todos los estudiantes.
3. **Condicional:** decide el estado académico.

El estudiante debería poder explicar qué ocurre en cada iteración.

11. Actividad de comprobación rápida

Durante los últimos minutos, pedir que respondan oralmente:

1. ¿Qué estructura usarían para coordenadas?
2. ¿Qué estructura usarían para eliminar duplicados?
3. ¿Qué estructura usarían para relacionar producto y precio?
4. ¿Qué diferencia fundamental existe entre tupla y conjunto?
5. ¿Qué representa una clave en un diccionario?
6. ¿Qué devuelve `items()`?

Una respuesta satisfactoria no debe limitarse a decir el nombre de la estructura. El estudiante debe justificarla.

12. Orientación para los ejercicios de casa

Los ejercicios están diseñados de manera progresiva:

- Ejercicios 1 y 2: consolidan tuplas.
- Ejercicios 3 y 4: consolidan conjuntos.
- Ejercicios 5 y 6: consolidan diccionarios.
- Ejercicio 7: trabaja selección conceptual.
- Ejercicio 8: integra conjuntos, diccionarios, ciclos y condicionales.

No es necesario resolver los ejercicios durante la clase. El objetivo es que sirvan como práctica posterior.

13. Cierre conceptual para el docente

Finalizar con una frase que los estudiantes puedan recordar:

La estructura de datos debe elegirse según la relación que existe entre los datos y las operaciones que necesitamos realizar.

Resumen:

Tupla	Secuencia ordenada e inmutable.
Conjunto	Colección de valores únicos y operaciones entre grupos.
Diccionario	Asociación entre claves y valores.

14. Nota metodológica

Es recomendable evitar introducir en esta sesión conceptos avanzados como hashabilidad, complejidad algorítmica formal, iteradores, comprensión de conjuntos/diccionarios o estructuras anidadas complejas. Aunque son temas relevantes, pueden distraer del objetivo principal de esta clase: **comprender las diferencias funcionales entre las estructuras y escoger la adecuada para problemas sencillos.**